Design and Timing Measurement of a Low-Power Sensor Data Logger on an STM32F103 with FreeRTOS

Saman Fuyal
Department of Electrical and Electronics Engineering
Kathmandu University
Dhulikhel, Nepal
smnfyl@gmail.com

***Abstract*—A data logger has to take samples at a steady rate, keep the correct time through a power cut, store data that can be checked later, and run for days from a battery. This report describes such a logger built on an STM32F103C8 microcontroller. It reads a BMP280 pressure and temperature sensor and an MPU6050 accelerometer and gyroscope over I2C, adds the chip's own temperature and supply voltage, and writes one CSV line per sample to a micro SD card with a CRC-32 at the end of every line. The firmware is written on the registers without a vendor library and runs four FreeRTOS tasks. Between samples the chip sleeps in STOP mode, and the real-time clock, running at 1024 Hz from a 32.768 kHz crystal, wakes it 1 ms before the next sample. In a 51-minute run at one sample per second the period measured by the firmware stayed within ±2 µs, a sample reached the card 28.5 ms after it was taken, and 3 154 lines had 0 CRC errors. STOP mode lowered the board current from 17.62 mA to 9.3 mA, a 47 % reduction. Comparing the time stamps with the line count showed that the real sample period is 1.0016 s, an error of 0.16 % that the firmware's own timing cannot see because it uses the same clock that schedules the samples. The final circuit runs from an 18650 cell on a matrix board with a charger, a boost converter and a coin cell for the clock.**

Keywords—data logger, STM32F103, FreeRTOS, tickless idle, STOP mode, real-time clock, SD card, FatFs, CRC-32, low power

I. Introduction

A data logger records measurements over a long time without a computer attached. For environmental and motion data this usually means a small board that runs from a battery, keeps the correct time through a power cut, and stores its data in a form that opens on any computer. Many such loggers have been published, most of them built on Arduino boards with an external real-time clock chip and an SD card [1], [2], [3].

This project builds a logger from cheap, easy-to-get modules around an STM32F103C8 microcontroller (the “Blue Pill” board) [4]. The aim was to understand every layer of it: the peripheral registers [5], a real-time operating system [6], the FAT file system on the card [7] and the battery supply. The logger had to:

- read temperature, pressure, acceleration and rotation at a fixed rate, 1 sample/s by default and settable from 250 ms to 60 s;
- stamp every sample with a clock that keeps running when the power is off;
- store each sample on a FAT-formatted micro SD card and lose as little as possible when the power fails or the card is pulled out;
- make every stored line checkable, so a damaged line can be found afterwards;
- use as little current as possible between samples and run from a rechargeable cell;
- start logging by itself at power-on, and offer a command line over USB for setting the clock, reading the sensors and measuring its own timing.

The contributions of this work are:

- a register-level FreeRTOS firmware in which a sensor task, a logger task and a command-line task share the buses through ordered mutexes, so slow SD card writes never delay a sample;
- tickless idle with STOP mode woken by a 1024 Hz real-time clock through EXTI events, which lowered the board current by 47 %;
- a CSV format with a CRC-32 on every line, checked by PC tools while logging and afterwards;
- timing measured on the target for every step of a sample, and a check of the sample period against the RTC time stamps, which found a 0.16 % period error that the on-target measurement could not see.

Section II reviews related loggers. Sections III and IV describe the architecture and the design, Section V how the logger was developed and tested, Section VI the results, and Section VII concludes.

II. Related Work and Research Gap

Baker built an open-source logger for environmental monitoring on an Arduino Mega, with a micro SD card on an Ethernet shield, a DS1307 clock backed by a CR2032 and a sleep library to save power [1]. Beddows and Mallon developed the Cave Pearl logger on an ATmega328P with a DS3231 clock [2]. It buffers readings in an EEPROM and writes them to the SD card in groups, checks the battery before each write, and sleeps at about 0.2 mA, which gives more than a year on three AA cells. Wickert et al. described Arduino-compatible loggers on the ATmega328p and 644p with a DS3231, an SD card and a watchdog; they sleep at about 12 µA and run for about two years on three AA cells [3].

More recent designs add a second place for the data. Santiago and Ciampitti built a multi-channel soil temperature

logger that stores data on a micro SD card and also sends it over LoRa to a database [8]. Perret et al. built eLogUp!, an Arduino MKR logger with a real-time clock, automatic wake-up and WiFi, GSM or LoRaWAN, powered by a LiPo cell with a solar panel [9]. TABLE I compares these designs with this work.

TABLE I. COMPARISON WITH PUBLISHED LOGGERS

Work	Microcontroller	Storage	Clock	Power	Line check	Software
[1]	ATmega2560	micro SD	DS1307 + CR2032	sleep library	none	Arduino loop
[2]	ATmega328P	micro SD, EEPROM buffer	DS3231	0.2 mA sleep, 3×AA	not reported	Arduino loop
[3]	ATmega328p/644p	SD card	DS3231	12 µA sleep, 3×AA	not reported	Arduino loop
[8]	Arduino + LoRa	micro SD + database	RTC module	battery	not reported	Arduino
[9]	Arduino MKR	radio upload	RTC, auto wake-up	LiPo + solar	not reported	Arduino
This work	STM32F103C8	micro SD (FatFs)	internal RTC, 1024 Hz, CR2032	STOP, 9.3 mA board, 18650	CRC-32 per line	FreeRTOS, registers

The published loggers reach very low sleep currents because their boards are designed for it. In every one of them the firmware is a single loop on an Arduino-class microcontroller, and as far as the papers describe, none stores a checksum with each line or measures its own sample period against its clock. This work uses ready-made modules, so its sleep current is set mostly by the modules, but it adds an operating system that keeps sampling independent of storage, a check on every stored line, and measurements of its own timing from two separate clocks.

III. SYSTEM ARCHITECTURE

Fig. 1 shows the logger. The STM32F103C8 is an Arm Cortex-M3 with 64 KB of flash and 20 KB of RAM [4], [10]. It runs from its internal 8 MHz RC oscillator (HSI). A faster clock is not needed, because the work per sample is small, and the HSI is running again immediately after STOP mode, which matters when the chip wakes up every second. The 32.768 kHz crystal drives the real-time clock (RTC), and a CR2032 on the VBAT pin keeps the RTC and its backup registers running when the main supply is off.

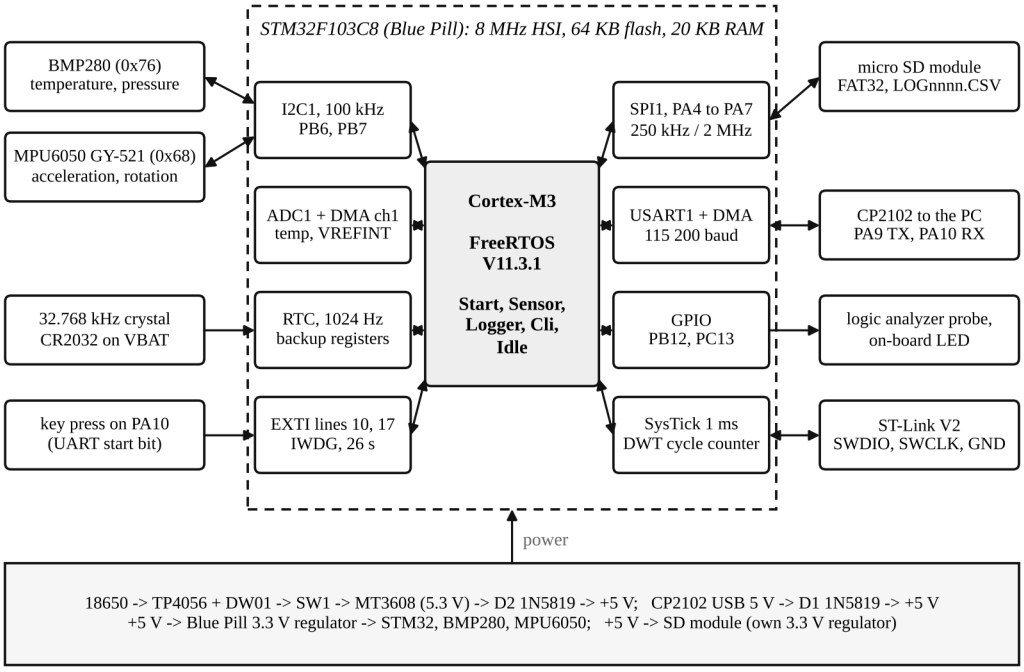


Fig. 1. System architecture. The dashed line is the microcontroller; the boxes inside it are the peripherals the firmware uses.

Both sensors share the I2C bus at 100 kHz. The BMP280 measures temperature and pressure [11]; the MPU6050 on a GY-521 module measures acceleration and rotation [12], [13]. The SD card is used in SPI mode [14]. A CP2102 USB-to-UART bridge connects the board to a PC for commands and live data [15]. TABLE II lists the parts and TABLE III the pins.

TABLE II. MAIN PARTS

Part	Function	Interface
STM32F103C8T6 (Blue Pill)	Cortex-M3, 64 KB flash, 20 KB RAM	SWD for programming
BMP280 module (4 pins)	temperature and pressure	I2C, address 0x76
GY-521 (MPU6050)	3-axis accelerometer and gyroscope	I2C, address 0x68
micro SD card module	storage, own 3.3 V regulator and level shifter	SPI, 5 V supply
CP2102 module	USB to serial for commands and live data	UART, 115 200 baud
18650 cell, TP4056 + DW01	battery, charger with protection	USB-C charging
MT3608 module	boost converter, set to 5.3 V	SW1 in its input

Part	Function	Interface
2 × 1N5819	Schottky diodes joining battery and USB supply	D1 USB, D2 boost
CR2032 in a holder	keeps the RTC and backup registers alive	VBAT pin
2 × 4.7 kΩ	I2C pull-ups	SCL, SDA to 3.3 V

TABLE III. MICROCONTROLLER PINS

Signal	Pin	Connected to
UART TX / RX	PA9 / PA10	CP2102 RXD / TXD; a falling edge on PA10 also wakes the chip from STOP
I2C SCL / SDA	PB6 / PB7	BMP280 and MPU6050, 4.7 kΩ to 3.3 V
SPI CS / SCK / MISO / MOSI	PA4 / PA5 / PA6 / PA7	SD card module
LED	PC13	on-board LED, active low
Test point	PB12	high while the sensors are read
Clock battery	VBAT	CR2032 positive terminal
Debug	SWDIO, SWCLK	ST-Link V2 (ground only, never its 3.3 V)

The power path is drawn at the bottom of Fig. 1. The 18650 cell connects only to the TP4056 module, which charges it from USB-C and disconnects it when it is over-discharged or shorted, through its DW01 protection chip [16]. From the module's output the current goes through the switch SW1 to the MT3608 boost converter [17], set to 5.30 V. The Schottky diode D2 after the converter leaves about 5.0 V on the board's 5 V line. The CP2102's USB 5 V reaches the same line through D1, so the board runs from either source and neither can push current into the other. With both connected, the battery side is higher (about 5.0 V against 4.7 V) and supplies the board. The Blue Pill's 3.3 V regulator supplies the microcontroller and both sensors; the SD module takes 5 V because it has its own regulator. The board ground is the TP4056's output ground, after the protection switch, not the cell's negative terminal. The full schematic, drawn in EasyEDA, is in the hardware folder of the repository.

IV. DESIGN METHODOLOGY

A. Firmware Structure

The firmware is 5 176 lines of C in 38 files (19 .c and 19 .h), plus FatFs R0.16 [7] and the FreeRTOS kernel V11.3.1 [6]. Every peripheral is set up through its registers [5]; the addresses and bit names are collected in board.h. Drivers return error codes and never print; only the logger and the command line do. Built with STM32CubeIDE, -Os, the program uses 44.8 KB of the 64 KB of flash and 14.4 KB of the 20 KB of RAM, 10 KB of which is the FreeRTOS heap. Fig. 2 shows the layers and TABLE IV the tasks.

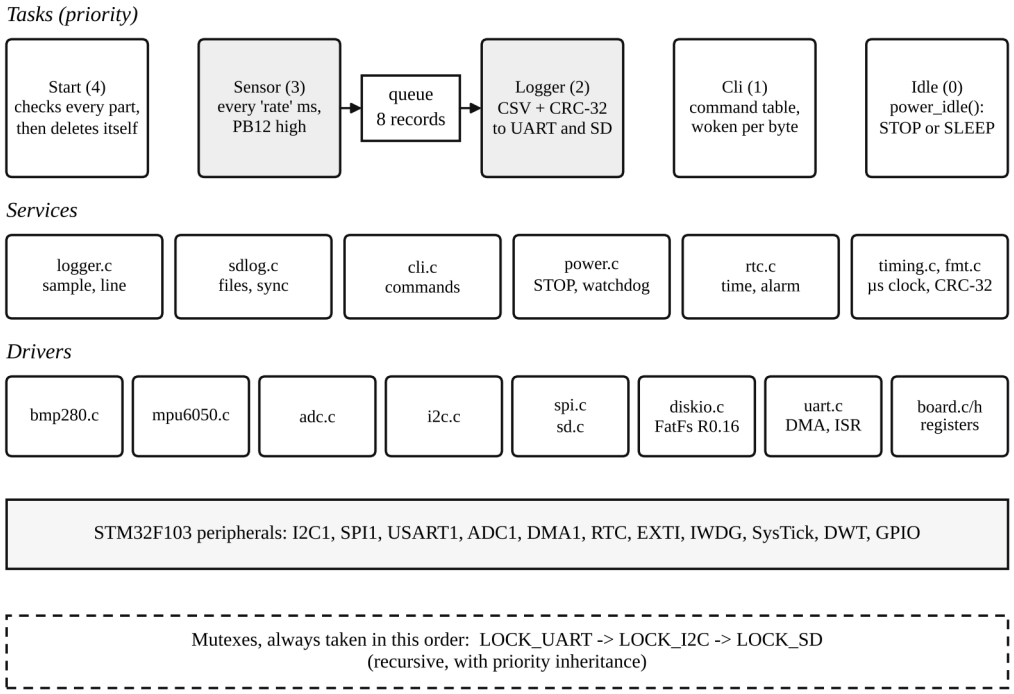


Fig. 2. Firmware layers. The sensor task and the logger task are joined by a queue; the three mutexes protect the UART, the I2C bus and the SD card.

TABLE IV. FREERTOS TASKS

Task	Priority	Stack	Job
Start	4	2 KB	checks every part at power-on, creates the other tasks, deletes itself
Sensor	3	1 KB	reads the sensors every ‘rate’ ms and puts a record in the queue
Logger	2	1.5 KB	turns a record into a CSV line with its CRC, prints it, writes it to the card
Cli	1	2 KB	the command line, woken by every received byte
Idle	0	kernel	FreeRTOS's own task; it puts the chip to sleep

The sensor task has the highest priority so that samples are taken on time even while the SD card is slow; a write that makes FatFs sync the file can take tens of milliseconds. The queue between the sensor task and the logger task holds eight records, eight seconds at the default rate. The next sample time is planned from the previous planned time, not from when the task woke up, so small delays do not add up [18].

Three resources are shared between tasks, and each has a mutex: the UART output, the I2C bus with both sensor drivers, and the SPI bus with the SD card and FatFs. They are always taken in the order UART, I2C, SD, which rules out a deadlock. FreeRTOS mutexes have priority inheritance [18]:

if the command line holds the I2C bus during the one-second gyro zeroing and the sensor task waits for it, the command line runs at the sensor task's priority until it lets go. The sensor and logger tasks take the UART only with a timeout. When the command line is printing, a log line still goes to the card and only its copy on the UART is skipped.

The UART sends through DMA channel 4 from two 128-byte buffers: one is sent while the other fills, so printing a log line does not hold up the task. Received bytes go from the interrupt into a FreeRTOS stream buffer, where the command-line task waits for them. Interrupts that call FreeRTOS functions must have a priority between 5 and 15 on this port; after reset every interrupt has priority 0, so the UART and DMA interrupts are set to 6 [19]. With the 8 MHz clock the baud rate register is set by (1). The real rate is 8 000 000/69 = 115 942 baud, 0.64 % above the nominal rate and well inside what a UART receiver accepts.

$$\mathrm{BRR} = \frac{f_{\mathrm{PCLK}}}{B} = \frac{8\,000\,000}{115\,200} = 69.4 \;\rightarrow\; 69 = 0\mathrm{x}45 \tag{1}$$

B. Timekeeping

The RTC normally counts seconds. Here its prescaler is set to 31, so the counter advances 1024 times per second, as in (2). The time in seconds is a base value kept in two backup registers plus the counter divided by 1024. Every 24 days the whole seconds are moved from the counter into the base so the counter cannot overflow. A magic value in a third backup register tells the firmware at power-on whether the time survived on the coin cell.

$$f_{\mathrm{RTC}} = \frac{32\,768\ \mathrm{Hz}}{\mathrm{PRL}+1} = 1024\ \mathrm{Hz}, \qquad t = t_{\mathrm{base}} + \frac{\mathrm{CNT}}{1024} \tag{2}$$

C. Low-Power Operation

FreeRTOS knows when the next task is due. When no task is ready, the idle task calls power_idle() with the time until then, called tickless idle [6], [18]. Fig. 3 shows what that function does. Three modes can be chosen with the power command. In run mode the CPU spins. In sleep mode it executes WFI: the CPU clock stops, the peripherals and the 1 ms tick keep running. In STOP mode, the default, every clock stops except the 32 kHz one [5], and the RTC alarm wakes the chip 1 ms before the next task is due.

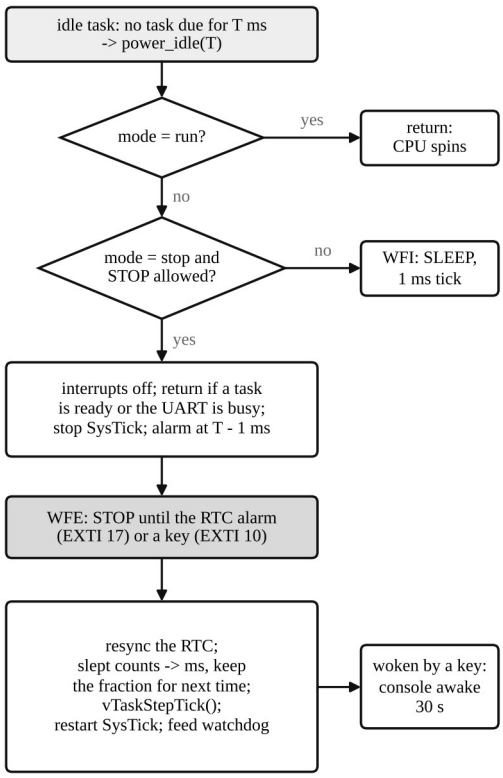


Fig. 3. Decisions in power_idle(), called by the idle task when no task is ready.

After waking, the number of RTC counts slept, n, is turned into milliseconds and added to the FreeRTOS tick with vTaskStepTick(). Because one RTC count is 1000/1024 ms, the remainder c is carried from one wake-up to the next, as in (3), so no fractions of a millisecond are lost in this step.

$$c \leftarrow c + 1000\,n, \qquad \Delta_{\mathrm{ms}} = \left\lfloor \frac{c}{1024} \right\rfloor, \qquad c \leftarrow c - 1024\,\Delta_{\mathrm{ms}} \tag{3}$$

A falling edge on PA10, the start bit of a received byte, also wakes the chip, so a key press brings the console back. The first byte is lost because the UART had no clock, and after that the chip stays out of STOP for 30 s. Both wake-up sources are EXTI events used with WFE, not interrupts, so no interrupt handler is involved at all (Section VI explains why). STOP is used only when it is safe; otherwise the chip falls back to sleep mode. The RTC crystal must be running, no debugger may be attached, the UART must have finished sending, at least 10 s must have passed since reset so the ST-Link can still connect for flashing, and the idle time must be at least 10 ms. The independent watchdog, which has its own 40 kHz clock, is fed every time the idle task runs. If the chip ever stays stuck for about 26 s, the watchdog restarts it, the start-up message reports it, and the board uses sleep mode instead of STOP until the next power-on.

Fig. 4 puts the pieces together for one sample at one sample per second. The sensor task raises the test pin PB12, starts a BMP280 conversion and waits; the chip can enter STOP during that wait. The logger task then builds the line, hands it to the UART DMA and writes it to the card. While the UART is still sending, power_idle() does not enter STOP, so the chip sleeps with WFI until the line has left, and then stays in STOP until 1 ms before the next sample.

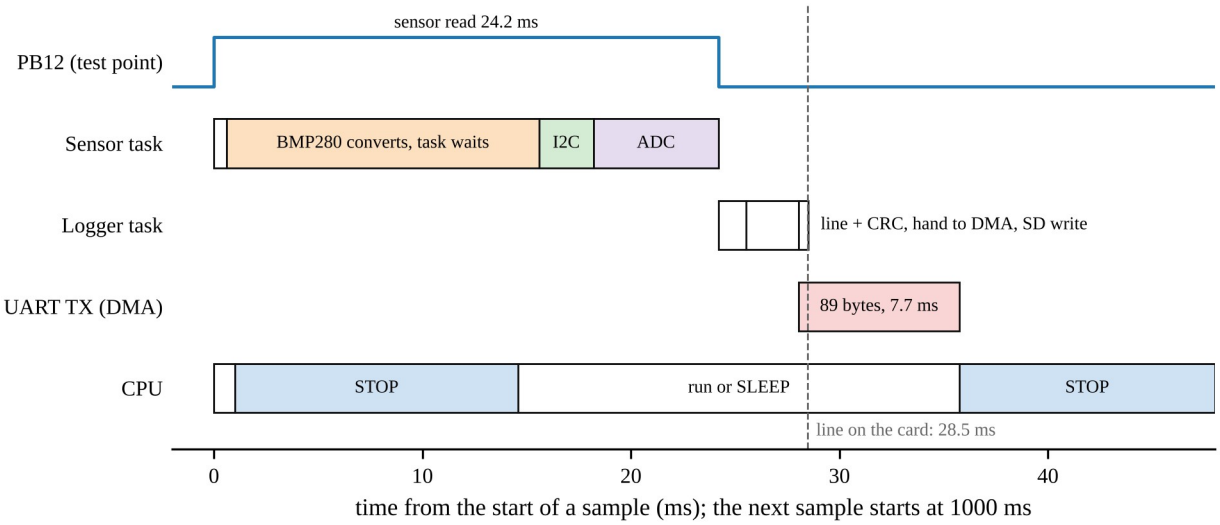


Fig. 4. One sample in time, drawn from the smallest step times measured by the firmware (TABLE VI). The CPU row follows the rules of power_idle().

D. Sensors and Calibration

The BMP280 runs in forced mode: the firmware starts one measurement, waits 15 ms while the sensor converts, and reads the result. Temperature and pressure are calculated with the integer compensation formulas from the datasheet and the 24 bytes of factory constants stored in each chip [11]. The selftest command runs the formulas on the datasheet's example and prints the expected 25.08 °C and 1006.53 hPa. Values outside the sensor's range are rejected and the read is tried once more.

The MPU6050 is set to ±2 g and ±250 °/s with its 44 Hz low-pass filter [13]. Acceleration and rotation are scaled by (4), where D is the raw reading, O the offset and S the counts per g. The accelerometer offsets and scale factors come from a six-position calibration (each axis pointing up and then down) and are stored in config.h: S = 16375, 16356 and 16510 counts per g for X, Y and Z, against the nominal 16 384. The gyro offsets are measured at every power-on while the board is still for one second; 131 counts per °/s is the datasheet value [12]. If the MPU6050 loses power for a moment, it wakes up asleep with its settings lost and returns zeros; the driver notices fourteen zero bytes, sets the chip up again and reports it.

$$a_i = \frac{D_{a,i} - O_{a,i}}{S_{a,i}}\ \text{g}, \qquad \omega_i = \frac{D_{g,i} - O_{g,i}}{131}\ ^\circ\text{/s} \tag{4}$$

The ADC measures the internal temperature sensor and the internal reference VREFINT 32 times each, copied to memory by DMA. Because VREFINT is fixed, its reading gives the supply voltage by (5). The datasheet gives VREFINT as 1.20 V typical, but it differs from chip to chip [4]; on this board it was measured as 1.1695 V with the vddcal command against a multimeter reading of 3.22 V on the 3.3 V pin. The chip temperature is computed by (6) and (7) with the datasheet's 1.43 V at 25 °C and 4.3 mV/°C, where the sensor voltage uses the nominal 1.20 V for VREFINT and the offset T_{off} is 0 °C.

$$V_{DD} = V_{REFINT} \cdot \frac{4095 \cdot 32}{\sum D_{REF,k}} \tag{5}$$

$$V_{TS} = 1.20\ \text{V} \cdot \frac{\sum D_{TS,k}}{\sum D_{REF,k}} \tag{6}$$

$$T_{chip} = 25\,^\circ\text{C} + \frac{1.43\ \text{V} - V_{TS}}{4.3\ \text{mV/°C}} + T_{off} \tag{7}$$

E. Data Format and Storage

Each log file starts with a header line, followed by one line per sample (a long line is wrapped here after a comma):

```
time,temp_C,press_hPa,ax_g,ay_g,az_g,gx_dps,gy_dps,gz_dps,
  vdd_V,chip_C
2026-10-20 01:16:46,28.33,998.63,-0.010,0.001,1.001,0.00,
  0.14,-0.03,3.22,29.69*0DC35899
```

The eight hexadecimal digits after the asterisk are the CRC-32 of the text before it, the same CRC that Python's zlib.crc32 computes, so the PC tools can check every line. A 32-bit CRC detects every error burst up to 32 bits long and almost all longer ones [20]. All values are computed in integers (hundredths of a degree, pascals, thousandths of a g) and printed with a fixed number of decimals; the firmware uses no floating point. Each logging session gets a new file, LOG0001.CSV, LOG0002.CSV and so on. The file is synced every 10 lines or every 10 s, so a power cut loses at most the last few lines.

Logging starts by itself at power-on. If a write or a sync fails, for example because the card was pulled out, the logger carries on over the UART, blinks the LED and tries to mount the card again every 5 s. The card starts at 250 kHz, as the SD specification requires for initialization, and runs at 2 MHz afterwards [14]. When the card mounts again, logging to the card continues.

F. Fault Handling and Tools

A logger that restarts and keeps logging is more useful than one that hangs. A stack overflow, an exhausted heap, a failed assertion and a hard fault all print the reason with interrupts off and restart the chip. For a hard fault the address of the failing instruction, the link register and the fault status register are saved in backup registers and printed at the next start-up, with the name of the task that was running. Every I2C wait has a 25 ms timeout; if a sensor holds the data line low, the driver clocks SCL by hand up to nine times and sends a STOP. A debugging session can leave hardware breakpoints armed after a reset, which causes a hard fault once the debugger is gone, so the start-up code clears them when no debugger is attached.

Every command is one row in a table in cli.c with its name, its function, the locks it needs and its help text, so the help can never get out of date; the Appendix lists them. Four Python scripts run on the PC: settime.py sets the clock from the computer, live_plot.py plots the data while the board logs and saves every good line, getfile.py copies a log file from the card over the serial port, and log_check.py checks the CRC of every line, reports gaps in time and plots each column.

V. Verification Methodology

Each peripheral was first brought up alone in a small test program before it went into the logger. The programs are kept in the bringup folder in the order they were written (TABLE V). The logger then grew in four milestones, and each was checked before the next began.

TABLE V. Bring-Up Programs

No.	Program	What it established
01	blink	clock and GPIO registers, the PC13 LED
02	uart_echo	USART1 by polling
03	uart_rx_irq_ringbuffer	receive interrupt with a 64-byte ring buffer
04	uart_cli	a small command line, SysTick 1 ms timing
05	i2c_sensor_id	I2C1, reading the BMP280 chip ID (0x58)
06	bmp280	temperature and pressure with the compensation formulas
07	mpu6050	acceleration and rotation, scaling, gyro zero
08	mpu6050_calibration	six-position accelerometer calibration
09	sd_card_read	SPI1, SD card start-up in SPI mode, reading sectors

M0 split the bring-up code into modules with a Makefile. M1 added FatFs, the CSV format with its CRC, and recovery from a removed card. M2 moved everything onto FreeRTOS with tasks, a queue, mutexes and DMA for the UART. M3 added tickless idle, STOP mode, the RTC alarm and the current measurements. All four were tested on a breadboard powered from the CP2102. At every power-on the start task checks each part and prints the result with the reset cause.

Timing was measured on the target. The firmware keeps a microsecond clock built from the FreeRTOS tick count and the SysTick counter, which counts down from 7999 to 0 in each millisecond. For each step of a sample it records the last, the smallest and the largest time, and the timing command prints them. The tasks command prints the share of CPU time per task, counted by the DWT cycle counter. Because the microsecond clock is driven by the same tick that schedules the samples, it cannot show an error in the tick itself. The time stamps in the log come from the RTC crystal instead, so the period between them, (8), is a second and independent measure, where t_1 and t_N are the first and last of N stamps.

$$\bar{P} = \frac{t_N - t_1}{N - 1} \tag{8}$$

Current was measured with a multimeter in series with the 5 V input of the breadboard. Before each reading the ST-Link was unplugged, because the chip never enters STOP with a debugger attached, and nobody typed for 30 s (Fig. 5). The battery life follows from (9), where C is the cell capacity, V_{bat} its voltage, η the boost converter efficiency and I_{in} the current at the input voltage V_{in}.

$$t_{bat} = \frac{C \cdot V_{bat} \cdot \eta}{V_{in} \cdot I_{in}} \tag{9}$$

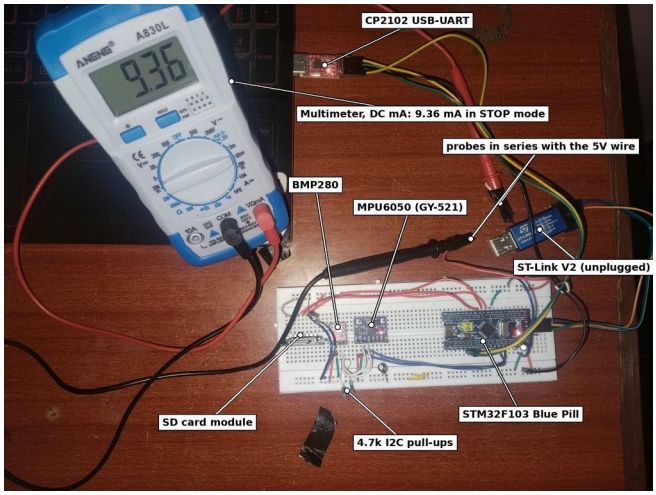


Fig. 5. Current measurement at the 5 V input of the breadboard in STOP mode.

Before the circuit went onto the matrix board, each power part was tested on its own on 28-10-2026: the 18650 cell, charging through the TP4056 with its protection, the switch, the MT3608 adjusted to 5.30 V before anything was connected to it, both diodes, the change-over between USB and battery, charging while the board ran, and the coin cell. The resistance from VBAT to 3.3 V was also measured and found open; on some Blue Pill copies these two pins are joined, and the 3.3 V supply would then charge the CR2032. The finished board was checked in stages: first the power path without any module, then the Blue Pill alone, then one module at a time.

VI. Results and Discussion

A. Timing

TABLE VI shows the step times after 51 minutes of logging at one sample per second with STOP mode on (20-10-2026, 3 056 samples).

TABLE VI. Step Times from the Timing Command

Step	Last (µs)	Min (µs)	Max (µs)
sample period	1 000 000	999 998	1 000 002
sensor read	24 198	24 197	24 198
line + CRC	1 343	1 318	1 408
UART (DMA)	99 368	2 524	99 368
SD write	5 261	261	41 815
sample -> SD	130 356	28 487	130 356

The 'last' and 'max' values of UART and sample -> SD were taken while typing 'timing': the CLI holds the UART, the logger waits up to 100 ms and then skips the UART copy of that line. 8 lines were skipped in 51 minutes. When nobody types, a line reaches the card 28.5 ms after the sample started.

The sensor read takes 24.2 ms and varies by only 1 µs. About 15 ms of it is the BMP280 conversion and about 6 ms the ADC, which is reset, calibrated, used and switched off again for every sample. An SD write takes 0.26 ms when FatFs only copies the line into its buffer and up to 42 ms when it writes a sector to the card. Of the time the CPU was awake, the idle task had 81.0 %, the logger 10.6 %, the sensor task 8.0 % and the command line 0.1 %. The lowest free heap was 1 520

bytes of 10 240, and 0 records were dropped from the queue. An earlier run at 250 ms per sample (822 samples) gave periods between 249 998 and 250 002 µs.

B. Data Integrity and Sensor Readings

The log LOG0123.CSV runs from 01:16:46 to 02:09:24 on 20-10-2026. It has 3 154 data lines, 0 with a wrong CRC, and no gap longer than 2 s. The values change very little over the run, so they mainly show the noise and offsets of the sensors (TABLE VII and Fig. 6).

TABLE VII. VALUES IN LOG0123.CSV (3 154 LINES)

Column	Min	Max	Mean	Std. dev.
BMP280 temperature (°C)	27.76	28.35	27.989	0.137
pressure (hPa)	998.28	998.76	998.538	0.094
acceleration X (g)	-0.061	0.010	-0.0060	0.0024
acceleration Y (g)	-0.010	0.008	0.0002	0.0011
acceleration Z (g)	0.987	1.005	0.9969	0.0019
rotation X (°/s)	-0.10	0.25	0.059	0.046
rotation Y (°/s)	-0.51	0.32	0.152	0.052
rotation Z (°/s)	-0.15	10.32	0.019	0.187
supply VDD (V)	3.22	3.22	3.220	0.000
chip temperature (°C)	28.72	29.74	29.058	0.154

The Z acceleration averages 0.997 g with a spread of 0.0019 g, which shows that the six-position calibration holds. The gyro reads close to zero; the largest value, 10.32 °/s on Z, is a single sample. The chip temperature reads 1.1 °C above the BMP280 on average. The supply voltage stayed between 3.22 and 3.22 V for the whole run, matching the multimeter.

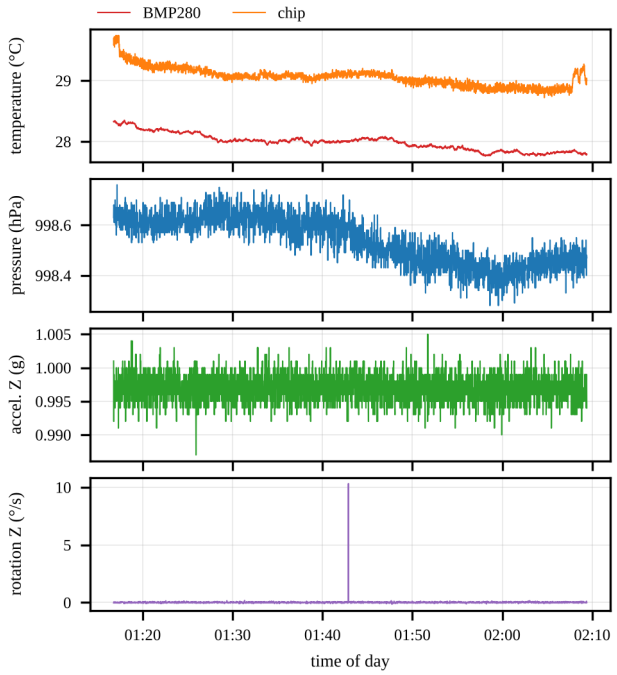


Fig. 6. Temperature, pressure, Z acceleration and Z rotation in LOG0123.CSV. All ten columns are plotted by log_check.py.

C. Sample Period Against the Real-Time Clock

The log spans 3158 s but has 3 154 lines, and 5 times the time stamp jumps by 2 s instead of 1 s, after data lines 74, 725, 1376, 2062, 2749. The jumps come at regular intervals, so this is a steady drift and not lost samples. By (8) the real sample period is 1.0016 s, 1.6 ms or 0.16 % longer than intended (Fig. 7). The timing command reported every period within 2 µs of 1 s, because it measures with the same FreeRTOS tick that schedules the samples.

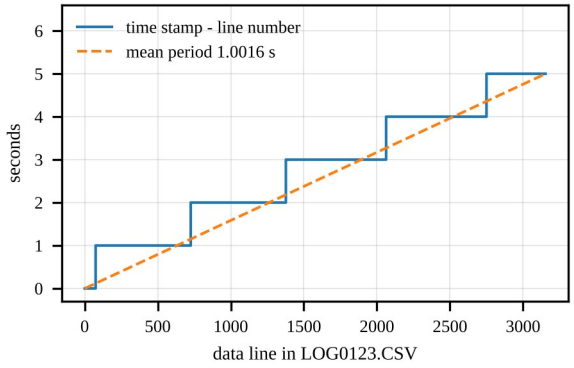


Fig. 7. Time stamp minus line number in LOG0123.CSV. Each step is a second without a sample; the dashed line is the mean period.

The time stamps come from the RTC crystal; it is the scheduler's time that runs slow. During STOP the FreeRTOS tick does not count, and power_idle() adds the time slept, measured with the RTC, when the chip wakes up. Three effects there can make the tick differ from the RTC. SysTick is stopped part-way through a millisecond and restarted from zero, so the part it had already counted is dropped at every STOP, which makes the tick slow. The RTC reading taken before STOP falls at a random point inside a 1/1024 s count, while the wake-up comes exactly on a count, so the time slept is overestimated by up to one count, which makes the tick fast. And the time awake is counted by the 8 MHz RC oscillator, which is accurate to about 1 % [4]. The first two depend on where in the millisecond the chip goes to sleep, so they do not cancel in general; the log shows the net effect.

D. Current Consumption

TABLE VIII gives the current of the whole breadboard (Blue Pill, both sensors and the SD module) at the 5 V input, measured on 19-10-2026.

TABLE VIII. BOARD CURRENT AT 5 V, LOGGING AT 1 SAMPLE/S

Command	State	Current (mA)
power run	logging, CPU never sleeps	17.62
power sleep	logging, WFI between tasks	12.94
power stop	logging, STOP between samples	9.3
stop + power stop	not logging, STOP	9.1
as above + mpusleep	not logging, MPU6050 asleep	7.92
power stop, no SD card	logging to the UART only	9.9

STOP mode cuts the board current by 47 %. The power command reported the chip in STOP 96 % of the time when nobody types. The microcontroller itself draws only tens of microamps in STOP [4], so most of the remaining 9.3 mA is the modules: the SD module's regulator, the power LEDs, the regulators and the MPU6050, whose share shows in the difference between the last two STOP rows, 1.2 mA. The PC13 LED takes about 1.7 mA, which is why in STOP mode it only flashes during a sample. Without a card the current is higher, because the LED blinks and the logger tries to mount the card every 5 s.

For a 2500 mAh 18650 at 3.7 V through a boost converter of about 85 % efficiency, (9) gives about 7.0 days of logging in STOP mode and 3.7 days in run mode.

E. Matrix Board

The circuit was rebuilt on a matrix board with every module on female headers (Fig. 8), finished on 31-10-2026. It runs from the 18650 alone or from the CP2102, charges through the TP4056's USB-C connector while it logs, and writes to the SD card in the same format as the breadboard. With the CR2032 fitted, the clock kept the correct time through a 2-minute power-off, so the time needs to be set only once.

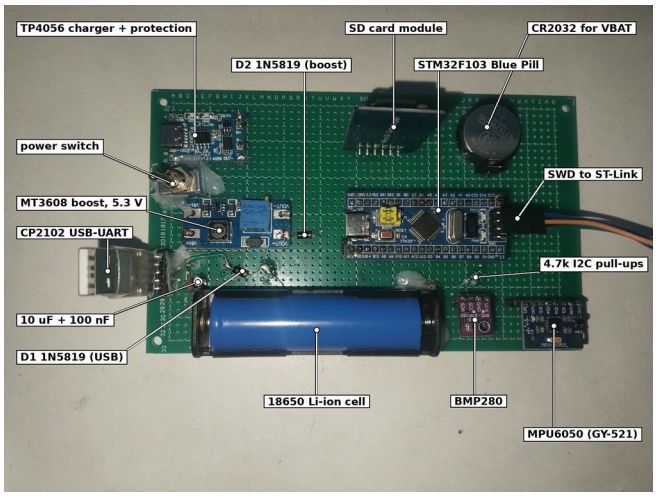


Fig. 8. Matrix board (rev B) running from the 18650 cell.

F. Problems Found and Fixed

STOP mode never woke up. The first version used the RTC alarm interrupt to wake the chip. It worked with the Makefile build but never woke up when built in STM32CubeIDE, and the watchdog reset the board every 26 s. The vector table in the compiled program showed the reason: CubeIDE's startup file names vector 41 RTCAlarm_IRQHandler, while the code defined RTC_Alarm_IRQHandler, so the alarm ran the default handler, an endless loop. The wake-up was changed to EXTI events with WFE. An event wakes the core without running any handler, so the code no longer depends on the handler names of a particular startup file.

The chip temperature and VDD froze. In an 18-minute log both stopped changing about two minutes after power-on. The ADC was converting continuously in the background, and after the first STOP it did not continue; restarting only the conversions gave nan values. Each reading now resets the ADC through the RCC, calibrates it, takes 32 pairs by DMA and switches it off again. This takes about 6 ms per sample and also lowers the current in STOP. The 51-minute log above was recorded after the fix.

VDD read 0.01 V low. The logged supply was 3.21 V while the multimeter showed 3.22 V. The internal reference was 1.1695 V instead of 1.20 V, and the value was cut off instead of rounded. The vddcal command now measures the reference against a meter reading, and VDD is rounded.

Flashing failed once the board slept. STM32CubeIDE reported “Error finishing flash operation.” The debug configuration was set to connect under reset, which needs the reset line of the ST-Link, and the three-wire cable does not have it. A software system reset fixed it; setting BOOT0 to 1 remains the fallback.

G. Discussion and Limitations

The cause of the 0.16 % period error is not confirmed. The three effects described above are candidates, and the log shows only their sum. The chip temperature uses the nominal 1.20 V for VREFINT, while VDD uses the measured 1.1695 V, and its offset is 0 °C; the datasheet allows 1.34 to 1.52 V at 25 °C [4], so the value shows changes in temperature better than the temperature itself. The battery life of 7.0 days is computed from the breadboard current at 5 V and an assumed 85 % boost efficiency. The sleep current is set by the modules, not by the microcontroller: the SD module's regulator, the power LEDs and the sensors stay powered.

VII. Conclusion

A sensor data logger was built on an STM32F103C8 with register-level drivers and FreeRTOS. It samples two sensors and the chip's own temperature and supply at a steady rate, stamps every line with a clock that survives a power cut, stores the data as CSV with a CRC-32 on every line, and recovers by itself from a removed card, a sensor that lost power and firmware faults. In a 51-minute test it logged 3 154 lines without a CRC error, a sample reached the card 28.5 ms after it was taken, and STOP mode reduced the board current by 47 % to 9.3 mA. Checking the scheduler against the RTC time stamps showed a sample period 1.6 ms too long, an error that the firmware's own measurement could not show. The circuit now runs from an 18650 cell on a matrix board.

References

[1] E. Baker, “Open source data logger for low-cost environmental monitoring,” *Biodiversity Data Journal*, vol. 2, Art. no. e1059, 2014, doi: 10.3897/BDJ.2.e1059.

[2] P. A. Beddows and E. K. Mallon, “Cave Pearl Data Logger: A flexible Arduino-based logging platform for long-term monitoring in harsh environments,” *Sensors*, vol. 18, no. 2, Art. no. 530, 2018, doi: 10.3390/s18020530.

[3] A. D. Wickert, C. T. Sandell, B. Schulz, and G.-H. C. Ng, “Open-source Arduino-compatible data loggers designed for field research,” *Hydrol. Earth Syst. Sci.*, vol. 23, pp. 2065–2076, 2019, doi: 10.5194/hess-23-2065-2019.

[4] STMicroelectronics, “STM32F103x8, STM32F103xB medium-density performance line Arm-based 32-bit MCU,” data sheet DS5319.

[5] STMicroelectronics, “RM0008 Reference manual: STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx advanced Arm-based 32-bit MCUs.”

[6] FreeRTOS, “FreeRTOS kernel V11.3.1.” [Online]. Available: https://www.freertos.org

[7] ChaN, “FatFs: Generic FAT filesystem module, R0.16.” [Online]. Available: http://elm-chan.org/fsw/ff/

[8] G. N. Santiago and I. Ciampitti, “Multiple channels, low-cost, and dual data storage data logger for building a soil temperature network,” *HardwareX*, Art. no. e00582, 2024, doi: 10.1016/j.ohx.2024.e00582.

[9] F. Perret *et al.*, “eLogUp! A precise, affordable and open-source IoT data logger to scale-up long-term environmental monitoring,” *HardwareX*, vol. 23, Art. no. e00660, 2025, doi: 10.1016/j.ohx.2025.e00660.

[10] J. Yiu, *The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors*, 3rd ed. Oxford, U.K.: Newnes, 2014.

[11] Bosch Sensortec, “BMP280 digital pressure sensor,” data sheet BST-BMP280-DS001.

[12] InvenSense, “MPU-6000 and MPU-6050 product specification,” PS-MPU-6000A-00.

[13] InvenSense, “MPU-6000 and MPU-6050 register map and descriptions,” RM-MPU-6000A-00.

[14] SD Association, “SD Specifications Part 1: Physical Layer Simplified Specification.”

[15] Silicon Labs, “CP2102/9 single-chip USB-to-UART bridge,” data sheet.

[16] NanJing Top Power, “TP4056 1A standalone linear Li-ion battery charger,” data sheet.

[17] Aerosemi, “MT3608 high efficiency 1.2 MHz 2 A step up converter,” data sheet.

[18] R. Barry, *Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide*. Real Time Engineers Ltd., 2016.

[19] STMicroelectronics, “PM0056 Programming manual: STM32F10xxx/20xxx/21xxx/L1xxxx Cortex-M3.”

[20] P. Koopman, “32-bit cyclic redundancy codes for Internet applications,” in *Proc. Int. Conf. Dependable Systems and Networks (DSN)*, Washington, DC, USA, 2002, pp. 459–468.

APPENDIX: FIRMWARE COMMANDS

TABLE IX. COMMANDS OF THE FIRMWARE

Command	Function
start, stop	start or stop logging (it starts by itself at power-on)
set rate N	N = 250 to 60 000 ms between samples
set time YYYY-MM-DD HH:MM:SS, time	set or show the clock
read, chip	one readable line from all sensors; supply and chip temperature
zero	measure the gyro offsets again (board still for 1 s)
status	logger state and error counters
ls, cat FILE, sd mount, sd init	the SD card: list, print a file, mount, start
tasks	FreeRTOS tasks: priority, state, free stack, CPU share
timing, timing reset	step times in microseconds
power, power run | sleep | stop	low-power mode and the time spent in STOP
power test, power reset	one 1 s STOP woken by the RTC alarm; clear the counters
scan, bmpread, mpuread, i2ctest N	I2C checks (debug build)
tscal, vddcal V	chip temperature offset; VREFINT against a meter (debug build)
selftest, crctest, mpusleep	BMP280 formulas, CRC-32, MPU6050 recovery (debug build)
sd read N, sd trace on | off	show a sector; trace the card start-up (debug build)